

Лекція 2. Сучасні стандарти верстки: HTML5 та CSS3

1. Вступ та еволюція вебверстки

Роль HTML та CSS у сучасних вебзастосунках

У сучасній веброзробці чіткий розподіл обов'язків між технологіями є фундаментальним принципом інженерного проектування. Для створення якісного вебінтерфейсу використовуються три ключові технології, які утворюють так звану «тріаду вебфронтенду»:

1. **HTML (HyperText Markup Language):** Відповідає виключно за **структуру та семантику** документа. Вона визначає, чим є кожен елемент на сторінці — заголовком, абзацом, кнопкою, таблицею, формою чи навігаційним меню.
2. **CSS (Cascading Style Sheets):** Відповідає за **візуальне представлення та дизайн**. Вона задає колірну гаму, шрифти, відступи, розміри, сітку розташування елементів, анімації та адаптивність під різні екрани.
3. **JavaScript:** Відповідає за **поведінку та інтерактивність**.

Головна парадигма сучасної верстки — **повне відділення вмісту (HTML) від його візуального оформлення (CSS)**. Історично верстальники часто змішували ці речі (наприклад, за допомогою атрибутів `color`, `align` або тегу `<font>`), що робило код неможливим для підтримки та масштабування. Сьогодні HTML-код має бути повністю очищений від стильової інформації, задаючи лише логічний каркас сторінки.

Еволюція стандарту HTML

Шлях розвитку мови розмітки пройшов кілька критичних етапів:

ЕВОЛЮЦІЯ СТАНДАРТУ HTML

HTML 4.01 (1999) -----> XHTML 1.0 (2000) -----> HTML5 (2014)
(Суворий синтаксис) (XML-сумісність, суворий) (Прагматичний, семантичний)

- **HTML 4.01 (1999):** Тривалий час залишався стандартом, проте не мав засобів для створення сучасних вебзастосунків (відсутність семантичних тегів для блоків, відсутність підтримки мультимедіа без сторонніх плагінів типу Flash).
- **XHTML 1.0 (2000):** Спроба консорціуму W3C перевести HTML на суворі рейки XML. Вимагала закриття всіх тегів (наприклад, `<img />`), нижнього регістру тегів та суворої синтаксичної дисципліни. Якщо у коді була помилка, браузер припиняв рендеринг сторінки. Через надмірну суворість технологія виявилася непрактичною для реального Вебу.
- **HTML5 (2014):** Результат об'єднання зусиль W3C та робочої групи WHATWG (Web Hypertext Application Technology Working Group). HTML5 повернувся до прагматизму:
 - **Філософія зворотної сумісності:** Браузери повинні коректно відображати навіть застарілий або частково некоректний код.
 - **Спрощення синтаксису:** Скасовано складні оголошення DOCTYPE (замість цього використовується лаконічний `<!DOCTYPE html>`).
 - **Семантична багаторазовість:** Впровадження нових тегів для чіткого опису структури сторінки.

- **Вбудоване мультимедіа:** Відмова від сторонніх плагінів (Flash, Silverlight) на користь нативних тегів `<audio>` та `<video>`.

Останнім офіційним монолітним стандартом HTML є **HTML5**, прийнятий консорціумом W3C у 2014 році. Після цього підхід до стандартизації кардинально змінився.

Сьогодні **HTML** — це **Living Standard (живий стандарт)**, який не має версій і постійно оновлюється в режимі реального часу.

- **Хто розробляє:** Робоча група **WHATWG** (Web Hypertext Application Technology Working Group), до якої входять розробники основних браузерів (Google, Apple, Mozilla, Microsoft).
- **Поточна назва:** Офіційно стандарт називається просто **HTML** (без цифр та версій).
- **Суть підходу:** Замість випуску великих оновлень (як HTML4 чи HTML5) нові функції, теги та API додаються до специфікації в міру їх розробки та впровадження у браузерах.

У 2019 році консорціум W3C офіційно передав контроль над стандартом HTML групі WHATWG, визнавши Living Standard єдиною офіційною версією.

Основна відмінність полягає в **моделі розвитку та версіонуванні**: HTML5 був фіксованим документом, випущеним у певний момент часу, тоді як WHATWG Living Standard — це безперервний процес оновлення без фіксованих версій.

Характеристика	HTML5 (Класичний)	WHATWG Living Standard
Організація	W3C (World Wide Web Consortium)	WHATWG (Google, Apple, Mozilla, Microsoft)
Формат	Статична специфікація (Snapshots: HTML5, HTML5.1, HTML5.2)	«Живий» стандарт, який оновлюється щодня
Версіонування	Має чіткі номери версій	Не має версій (офіційна назва — просто HTML)
Підхід до впровадження	Спочатку описується специфікація, потім реалізується в браузерах	Нові функції додаються за участю розробників браузерів у міру їх готовності
Статус	Застарілий підхід (W3C заклав свої версії HTML)	Єдиний офіційний стандарт HTML з 2019 року

Еволюція та модульність CSS3

На відміну від попередніх версій (CSS1, CSS2.1), які випускалися як єдині монолітні специфікації, **CSS3** розроблено за **модульним принципом**. Консорціум W3C розбив специфікацію CSS на десятки незалежних модулів, кожен з яких розвивається та оновлюється у власному темпі:

- **CSS Color Module:** Нові колірні моделі (HSL/HSLA, RGBA, CSS variables).
- **CSS Flexible Box Layout (Flexbox):** Одномірною модель верстки.
- **CSS Grid Layout:** Двомірною сітковою моделлю верстки.
- **CSS Transitions & Animations:** Створення плавності та анімацій без використання JavaScript.
- **CSS Media Queries:** Механізми для адаптації під мобільні пристрої.

2. Семантична розмітка HTML5 та мультимедіа

Концепція семантики

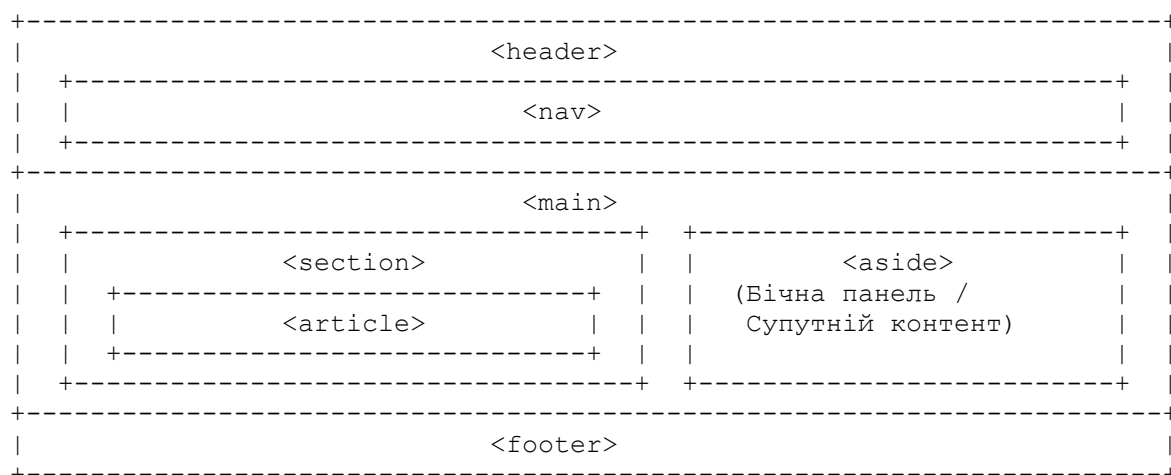
Семантика у вебі — це використання HTML-тегів відповідно до їхнього логічного значення та призначення, а не за їхнім візуальним виглядом.

У часи HTML4 більшість макетів будувалися за допомогою «DIV-супу»: сторінка складалася з десятків вкладених тегів `<div id="header">`, `<div class="menu">`, `<div class="content">`, `<div class="footer">`. Для браузерів та пошукових роботів усі ці елементи були абсолютно однаковими звичайними блоками.

Чому семантика є критично важливою?

1. **SEO (Search Engine Optimization):** Пошукові роботи (Googlebot тощо) краще розуміють структуру сторінки, виділяють головний контент від допоміжного, що безпосередньо впливає на ранжування сайту.
2. **Доступність (Accessibility):** Скрінрідери (програми для зчитування екрана невидючими та слабозорими людьми) використовують семантичні теги для швидкої навігації сторінкою (наприклад, дозволяють одразу перейти до блоку `<nav>` або `<main>`).
3. **Підтримка коду (Maintainability):** Читабельність коду для інших розробників суттєво зростає.

Основні семантичні теги HTML5



- **<header>**: «Шапка» сторінки або окремої секції. Зазвичай містить логотип, назву сайту, пошукову стрічку або елементи навігації.
- **<nav>**: Блок для головних навігаційних посилань (меню сайту, таблиця змісту).
- **<main>**: Унікальний основний контент документа. На сторінці має бути лише один тег **<main>**, який не повторюється на інших сторінках сайту.
- **<section>**: Тематична група контенту, яка зазвичай має власний заголовок (наприклад, «Про нас», «Послуги», «Контакти»).
- **<article>**: Самостійний, закончений блок інформації, який можна вирвати з контексту сторінки та поширити окремо (наприклад, стаття в блозі, новина, картка товару, коментар).
- **<aside>**: Блок із додатковим або супутнім контентом, що опосередковано пов'язаний з основним (бічна панель / sidebar, реклама, списки популярних статей).
- **<footer>**: «Підвал» сторінки або секції. Містить авторські права, контактну інформацію, посилання на політику конфіденційності.

Допоміжні семантичні теги:

- `<figure>` та `<figcaption>`: Контейнер для самостійного контенту (зображення, графік, код) та його підпису.
- `<time>`: Позначає дату або час у людино- та машиночитаному форматі (наприклад, `<time datetime="2026-09-15">15 вересня 2026 року</time>`).
- `<mark>`: Виділений або підсвічений текст (наприклад, результати пошуку).

Мультимедійні елементи та сучасна графіка

HTML5 ввів нативні засоби для роботи з медіаконтентом.

Робота з `<video>` та `<audio>`

Теги дозволяють відтворювати файли без зовнішніх плеєрів:

HTML

```
<video controls width="640" height="360" poster="preview.jpg" muted>
  <source src="video.mp4" type="video/mp4">
  <source src="video.webm" type="video/webm">
  Ваш браузер не підтримує елемент video.
</video>
```

- **Атрибути:**
 - `controls` — відображає нативну панель управління (play, pause, гучність);
 - `autoplay` — автоматичний старт (у більшості сучасних браузерів працює лише за наявності атрибута `muted`);
 - `loop` — зациклене відтворення;
 - `poster` — URL зображення, яке відображається до запуску відео.

Адаптивна графіка з тегом `<picture>`

Тег `<picture>` дозволяє віддавати різні зображення залежно від характеристик пристрою (роздільна здатність екрана, підтримка форматів):

HTML

```
<picture>
  <source srcset="image-desktop.webp" media="(min-width: 1024px)"
  type="image/webp">
  <source srcset="image-mobile.jpg" media="(max-width: 767px)">
  
</picture>
```

SVG (Scalable Vector Graphics) vs Растрова графіка

- **Растрова графіка (JPEG, PNG, WebP, AVIF):** Складається з сітки пікселів. При збільшенні втрачає якість.
- **Векторна графіка (SVG):** Описується математичними формулами (лінії, криві, багатокутники). Безкінечно масштабується без втрати чіткості, має малу вагу та може стилізуватися через CSS.

Сучасні вебформи у HTML5

HTML5 суттєво розширив можливості елемента `<input>`, що дозволило реалізувати первинну валідацію даних без мови JavaScript.

Нові типи введення (Input Types):

- `<input type="email">` — вимагає введення коректної адреси електронної пошти.
- `<input type="url">` — вимагає формату URL.
- `<input type="number" min="1" max="100" step="5">` — числове поле з контролем меж.
- `<input type="range">` — повзунок для вибору значення.
- `<input type="date">` / `time` / `datetime-local` — нативний календар та вибір часу.
- `<input type="color">` — нативна палітра вибору кольору.

Атрибути валідації на стороні клієнта:

- `required` — робить поле обов'язковим для заповнення.
- `placeholder` — підказка всередині поля.
- `pattern` — регулярний вираз для перевірки формату (наприклад, `pattern="^\+380\d{9}$"` для українського номера телефону).
- `autofocus` — автоматичне встановлення фокусу на поле при завантаженні.

3. Сучасний CSS3: Каскад, селектори та блокова модель

Фундаментальні механізми CSS

1. Каскадність та успадкування (Inheritance)

CSS розшифровується як **Cascading Style Sheets** (Каскадні таблиці стилів). **Каскад** — це алгоритм, за яким браузер вирішує, яке саме значення властивості застосувати до елемента, якщо на нього претендують декілька різних CSS-правил.

Джерела стилів (у порядку зростання пріоритету):

1. Стили за замовчуванням у браузері (User Agent Stylesheet).
2. Стили користувача (User Stylesheet).
3. Стили автора сайту (Author Stylesheet).

Успадкування визначає, чи передаються значення властивостей від батьківського елемента до дочірніх.

- *Успадковуються:* текстові властивості (`color`, `font-family`, `font-size`, `line-height`, `text-align`).
- *НЕ успадковуються:* блокові властивості (`margin`, `padding`, `border`, `width`, `height`, `background`).

2. Специфічність селекторів (Specificity)

Якщо до одного й того самого елемента застосовуються суперечливі правила з одного джерела, браузер обчислює **вагу (специфічність)** кожного селектора.

Формула розрахунку специфічності (система з 4-х категорій (A, B, C, D)):

- **Category A (Inline):** Стили, написані безпосередньо в HTML через атрибут `style="..."` ($Вага = 1000$).
- **Category B (ID):** Селектори за ідентифікатором, наприклад `#header` ($Вага = 100$).
- **Category C (Classes, Attributes, Pseudo-classes):** Класи (`.btn`), атрибути (`[type="text"]`), псевдокласи (`:hover`) ($Вага = 10$).

- **Category D (Elements, Pseudo-elements):** Теги (`h1`, `div`), псевдоелементи (`::before`) (`Bara = 1`).

ПРИКЛАДИ РОЗРАХУНКУ СПЕЦИФІЧНОСТІ:

```
1. div p                -> (0, 0, 0, 2) = 2 бали
2. .content .text       -> (0, 0, 2, 0) = 20 балів
3. #main-header .nav-item -> (0, 1, 1, 0) = 110 балів
4. <div style="color: red"> -> (1, 0, 0, 0) = 1000 балів
```

Примітка: Директива `!important` переважає будь-яку специфічність, проте її використання є поганою практикою, оскільки вона ламає природний каскад стилів.

Сучасні селектори, псевдокласи та псевдоелементи

Розширені псевдокласи стану та структури:

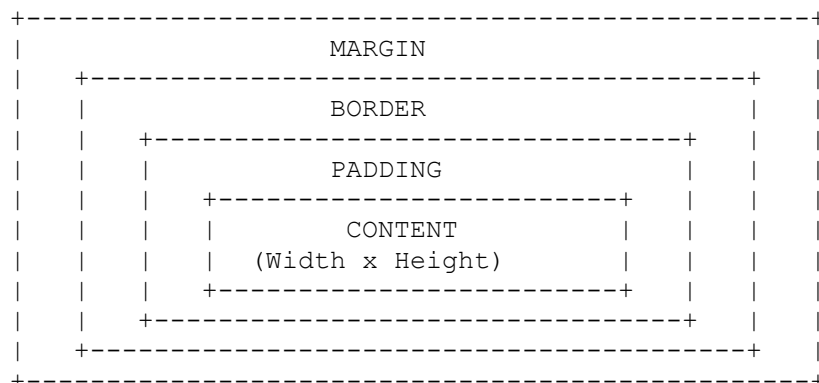
- `:hover`, `:focus`, `:active` — динамічні стани елементів.
- `:nth-child(n)` — вибір елемента за його порядковим номером (наприклад, `:nth-child(2n)` вибирає всі парні елементи).
- `:not(.active)` — заперечення (вибирає всі елементи, які не мають класу `.active`).
- `:is(h1, h2, h3)` та `:where(h1, h2, h3)` — групувальні селектори. Відмінність: `:is()` приймає специфічність найважчого селектора у списку, а `:where()` завжди має специфічність 0.

Псевдоелементи:

- `::before` та `::after` — дозволяють вставляти декоративний контент до або після вмісту елемента через CSS (вимагають наявності властивості `content: ""`).

Блокова модель (Box Model)

Кожен елемент у вебдокументі розглядається браузером як прямокутний блок (Box).



Складові Box Model:

1. **Content (Вміст):** Область, де розташовується текст або зображення (задається через `width` та `height`).
2. **Padding (Внутрішній відступ):** Простір між вмістом та рамкою блоку. Має тло елемента.
3. **Border (Рамка):** Лінія, що оперізує внутрішній відступ та вміст.
4. **Margin (Зовнішній відступ):** Простір між рамкою даного блоку та сусідніми елементами. Завжди прозорий.

Властивість `box-sizing`

За замовчуванням у браузерях використовується `box-sizing: content-box;`. При цьому загальна реальна ширина блоку на екрані розраховується за формулою:

$\text{Total Width} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right}$

Якщо ви задали `width: 300px`, `padding: 20px`, `border: 5px`, то реальна ширина елемента складе $300 + 40 + 10 = 350\text{px}$. Це дуже незручно при розрахунку макетів.

Сучасний стандарт розробки: Використання `box-sizing: border-box;`. При цьому значення `width` вже включає в себе `padding` та `border`. Загальна ширина залишиться рівною `300px`, а область вмісту автоматично стиснеться.

CSS

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

CSS-змінні (Custom Properties)

CSS-змінні дозволяють зберігати значення (кольори, шрифти, розміри) та повторно використовувати їх у документі.

CSS

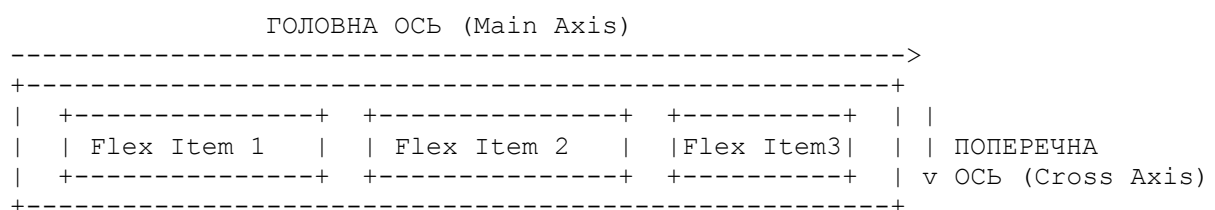
```
/* Оголошення глобальних змінних у кореневому елементі */
:root {
  --primary-color: #3498db;
  --base-padding: 16px;
  --font-stack: 'Roboto', sans-serif;
}

/* Використання */
.button {
  background-color: var(--primary-color);
  padding: var(--base-padding);
  font-family: var(--font-stack);
}
```

4. Гнучкі макети: Flexbox та CSS Grid

Flexbox (Flexible Box Layout) — Одномірна модель

Flexbox призначений для розкладки елементів уздовж **однієї осі** — або в рядок (горизонтально), або в стовпчик (вертикально).



Властивості Flex-контейнера (`display: flex`):

- `flex-direction`: Визначає напрямок головної осі (`row` — за замовчуванням, `row-reverse`, `column`, `column-reverse`).
- `justify-content`: Вирівнювання елементів уздовж **головної осі**:
 - `flex-start` / `flex-end` / `center`;
 - `space-between` (рівномірний простір між елементами);
 - `space-around` / `space-evenly`.
- `align-items`: Вирівнювання елементів уздовж **поперечної осі** (`stretch`, `flex-start`, `flex-end`, `center`).
- `flex-wrap`: Дозволяє перенесення елементів на новий рядок (`nowrap`, `wrap`).
- `gap`: Задає відстань між елементами (наприклад, `gap: 20px;`).

Властивості Flex-елементів:

- `flex-grow`: Коефіцієнт розтягування елемента, якщо в контейнері є вільний простір.
- `flex-shrink`: Коефіцієнт стискання елемента при нестачі простору.
- `flex-basis`: Базовий розмір елемента перед розподілом вільного простору.
- Скорочений запис: `flex: 1 0 200px;` (`flex-grow flex-shrink flex-basis`).
- `align-self`: Перевизначає `align-items` для конкретного дочірнього елемента.

CSS Grid Layout — Двовірна модель

CSS Grid призначений для розкладки елементів одночасно у **двох вимірах** — у рядки та стовпчики.

	Column 1	Column 2	Column 3
Row 1	Grid Cell	Grid Cell	Grid Cell
Row 2	Grid Cell	Grid Cell	Grid Cell

Ключові поняття Grid:

- **Grid Container & Grid Items**: Батьківський блок із `display: grid` та його безпосередні дочірні елементи.
- **Grid Lines (Лінії сітки)**: Горизонтальні та вертикальні розділові лінії (нумеруються починаючи з 1).
- **Grid Tracks**: Простір між двома сусідніми лініями сітки (стовпчик або рядок).
- **Grid Cell (Комірка)**: Найменша одиниця сітки на перетині рядка та стовпчика.
- **Одиниця `fr` (Fraction)**: Частка вільного простору контейнера (наприклад, `1fr 2fr` ділить простір у співвідношенні 1:2).

Базове оголошення сітки:

CSS

```
.grid-container {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 однакових стовпчики */
  grid-template-rows: 100px auto;      /* 1-й рядок 100px, 2-й за контентом */
  gap: 20px;                            /* Відступи у сітці */
}
```

Позиціонування елементів (Grid Placement):

CSS

```
.card-featured {
```

```

    grid-column: 1 / 3; /* Займає простір від 1-ї до 3-ї лінії (2 стовпчики) */
    grid-row: 1 / 2;
}

```

Шаблони поверхонь (grid-template-areas):

Дозволяє верстати макет за допомогою наочного текстового шаблону:

CSS

```

.site-layout {
    display: grid;
    grid-template-areas:
        "header header"
        "sidebar main"
        "footer footer";
    grid-template-columns: 250px 1fr;
}

.header { grid-area: header; }
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.footer { grid-area: footer; }

```

Порівняння Flexbox vs CSS Grid

Критерій	Flexbox	CSS Grid
Вимірність	Одномірна (Рядок АБО Стовпчик).	Двовірна (Рядки І Стовпчики одночасно).
Підхід	Content-first (розмір блоків залежить від контенту).	Layout-first (контент вкладається у заздалегідь створену сітку).
Типові кейси	Навігаційні меню, кнопки, картки, вирівнювання іконки з текстом.	Загальні макети сторінок, галереї товарів, складні дашборди.

5. Адаптивний дизайн (Responsive Web Design, RWD)

Принципи Responsive Web Design

Термін ввів Ітан Маркотт у 2010 році. **Адаптивний вебдизайн** — це підхід до верстки, який забезпечує коректне та зручне відображення вебсайту на пристроях з будь-яким розміром екрана (від смартфона до 4К-монітора).

Три стовпи RWD:

1. Гнучка сітка на відсотках / fr (Fluid Grid).
2. Гнучкі зображення та медіаконтент (Fluid Images — `max-width: 100%`).
3. Медіа-запити (Media Queries).

Метатег Viewport

Без цього тегу мобільні браузері емулюють широкий екран настільного комп'ютера (зазвичай 980px) і масштабують сторінку так, що текст стає мікроскопічним.

Обов'язковий тег у `<head>` кожної сторінки:

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- `width=device-width` — встановлює ширину області перегляду рівною фізичній ширині екрана пристрою.
- `initial-scale=1.0` — встановлює початковий масштаб 100%.

Медіа-запити (Media Queries)

CSS-механізм, який дозволяє застосовувати блоки стилів тільки при виконанні певних умов (наприклад, ширини екрана).

CSS

```
/* Базові стилі для десктопа */
.container {
  width: 1200px;
}

/* Стилi для планшетiв та мобiльних пристроїв */
@media (max-width: 768px) {
  .container {
    width: 100%;
    padding: 0 15px;
  }
}
```

Стратегія Mobile-First vs Desktop-First

СТРАТЕГІЇ ВЕРСТКИ

Desktop-First (max-width)	Mobile-First (min-width)
----->	----->
[Desktop] -> [Tablet] -> [Mobile]	[Mobile] -> [Tablet] -> [Desktop]

1. Desktop-First:

Розробка починається з повної версії для ПК, а потім стилі перевизначаються та «урізаються» для менших екранів за допомогою (`max-width: ...`).

2. Mobile-First:

Розробка починається з найпростішого макета для мобільного пристрою. У міру збільшення екрана додаються нові колонки та складні елементи за допомогою (`min-width: ...`).

CSS

```
/* Mobile-First: Базові стилі для мобільних */
.card-list {
  display: flex;
  flex-direction: column;
}

/* Додаємо 2 колонки для планшетів */
@media (min-width: 600px) {
  .card-list {
    flex-direction: row;
    flex-wrap: wrap;
  }
}

/* Додаємо Grid-сітку для десктопа */
@media (min-width: 1024px) {
  .card-list {
    display: grid;
    grid-template-columns: 1fr 1fr;
  }
}
```

```
.card-list {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
}
```

Переваги Mobile-First: Оптимізація продуктивності для мобільних пристроїв (мобільний браузер не завантажує зайві стилі), кращий UX та відповідність сучасній вимозі Google Mobile-First Indexing.